

AI Trust Decision Brief

Student: Nandini Chaurasiya

Assigned Pack: D5-B — Payment Webhook AI Plan

1. Problem Summary

FlashCart runs limited drops where hundreds of people compete for 100–300 units at once. The old flow only reduced inventory *after* a payment webhook arrived. Two customers could hit Buy Now at the same millisecond, both sessions proceed to payment independently — no lock stops them — and both webhooks fire for stock that only existed once. A customer refreshing the page or hitting Back and retrying creates the same race.

This caused negative inventory, duplicate orders, double charges, and support with no way to explain order states.

The AI plan tries to fix webhook reliability. Some ideas are correct. Two recommendations will cause the exact same crashes again if shipped as written.

2. AI Recommendation Decision Table

#	Recommendation	Decision	Why	Failure If Wrong	Safer Action
1	Trust webhook as order truth	Block	No reservation lock before payment. Two customers proceed independently. Customer refresh creates same race.	Two webhooks create two orders. Inventory goes negative. One customer cancelled after payment.	Reserve inventory at checkout start with a DB lock. Webhook confirms payment only.
2	Store gateway event IDs	Modify	Good idea. But SELECT-then-INSERT without a DB constraint is not race-safe. Two webhooks	Both read "not seen", both INSERT, duplicate order created.	Add UNIQUE(event_id). Use INSERT ... ON CONFLICT DO NOTHING inside a transaction.

			both pass the check before either commits.		
3	Retry with exponential backoff	Trust	Safe once idempotency is fixed. Delays (5s/30s/2min) are reasonable. Re-running an idempotent handler causes no new harm.	Only dangerous if enabled before idempotency is proven — a partial failure retried re-runs its side effects.	Ship after UNIQUE(event_id) is confirmed in production.
4	Queue webhooks async	Needs Proof	Only justified if gateway timeouts are real and measured. Proof threshold: production logs showing p95 webhook processing above 3 seconds during a drop. Below that, synchronous is safer.	Queue failures are invisible. Events drop or double-process. Small team now operates a queue under drop load.	Show latency logs first. Define the number that flips this from Needs Proof to justified.
5	Ignore out-of-order events	Block	Most dangerous. Creates orders for payments that may be voided. Assumes gateway's last event is always	payment_failure arrives, checkout marked failed, delayed payment_success arrives, AI creates order, payment was cancelled,	Closed state machine. failed → paid is not legal. Flag success-after-failure for human review.

			authoritative — FlashCart cannot verify this.	goods ship for free.	
6	payment_id as only idempotency key	Block	Two concurrent webhook deliveries share the same payment_id. SELECT- before- INSERT is not atomic without a DB constraint.	Two simultaneous webhooks both pass the check at the same millisecond. Two orders, inventory deducted twice. Exact repeat of the last incident.	Use event_id as the unique key. Enforce with UNIQUE(event_id) at DB level, not app level.

3. Most Dangerous Recommendation

Recommendation 5 — Ignore out-of-order events.

Why it sounds correct: Out-of-order webhook delivery is a real problem in distributed systems. Having a rule for it sounds senior. A junior engineer trusts it because it uses real vocabulary and covers an edge case most tutorials skip. The flaw: it assumes the gateway's last event is always the truth — but FlashCart cannot verify payment settlement without checking directly.

Exact failure timeline:

First: Customer pays. Gateway has a network hiccup.

Sends payment_failure first.

Then: Backend marks checkout failed.

No order. No inventory reserved.

Then: Gateway retries 90 seconds later.

payment_success arrives late.

AI Rec 5 says: update order to paid.

Then: Backend creates order, reduces inventory.

Production breaks because:

The payment may have been voided or never settled.

FlashCart creates an order with no valid payment.

Goods ship for free. Inventory goes negative.

Customer/support impact:

Customer gets confirmation email, expects delivery.

Support sees failure then success — no clear truth.

Cannot explain if order is real.

Same scenario as the last public apology.

What to do instead: Closed state machine. pending → paid is legal. pending → failed is legal. failed → paid is not legal — it needs a new payment attempt, not a webhook flip. Any success-after-failure is flagged for human review, never auto-processed.

4. Useful But Incomplete Recommendation

Recommendation 2 — Store gateway event IDs.

What was useful: Correct foundation. Recording processed event_id values is exactly the right approach to idempotency.

What was missing: The SELECT-then-INSERT without a DB constraint is not race-safe. Two concurrent webhooks both SELECT "not found", both INSERT — the database never stops them.

The fix: Add UNIQUE (event_id) to payment_events. Use INSERT ... ON CONFLICT DO NOTHING. Check rows affected — if 0, exit immediately. The DB constraint is the actual guard. The SELECT is just observability.

5. One Recommendation Trusted For V1

Recommendation 3 — Retry with exponential backoff.

Delays of 5s / 30s / 2min are reasonable. The problem it solves — a transient DB hiccup silently losing a webhook — is real.

Why it is safe: Once the handler is idempotent, retrying it causes no new harm. Re-running a side-effect-free operation is safe by definition.

Boundary that must hold: Only enable retries after `UNIQUE(event_id)` is confirmed working in production. If retries run before idempotency is proven, a partially-failed handler re-runs its side effects — the exact duplicate order problem being fixed.

6. Safer V1 Plan

Inventory ownership: Reserved at checkout start, not at webhook. No reservation = no order, regardless of what the webhook says.

Checkout state machine: created → reserved → paid → fulfilled. Also expired and failed. No skipping. failed → paid is blocked.

Payment state: Stored in `payment_events` with `UNIQUE(event_id)`. Webhook updates payment, then triggers a state transition only if it is legal.

Order confirmation rule: Order created only when checkout is reserved AND webhook confirms paid. Both must be true simultaneously.

Duplicate handling: `UNIQUE(event_id) + INSERT ... ON CONFLICT DO NOTHING + transaction`. If rows affected = 0, exit immediately.

Expiry: Cron job every 15 minutes. 15 minutes is chosen because it is long enough for a customer to handle 3DS authentication or a card decline, but short enough that inventory is not locked for an entire drop window. Any reserved checkout with no payment after 15 minutes transitions to expired and releases inventory. If the cron job misses a run, the next run catches all overdue checkouts because the query is time-based, not event-triggered.

Support visibility: Every checkout has status and `failure_reason`. Support queries any `checkout_id` and sees full state history. Customers see clear messages: "reservation expired", "payment failed", "out of stock."

7. State / Side Effect Protection

What must not happen twice: Inventory deduction for one checkout. Order creation for one payment event.

Two separate problems — two separate guards:

Guard 1 — event-level idempotency: `UNIQUE(event_id)` on `payment_events` stops the *same webhook* from being processed twice. Gateway sends a duplicate — second `INSERT` hits the constraint, returns 0 rows affected, handler exits. Same event cannot cause two orders.

Guard 2 — operation-level idempotency: `SELECT ... FOR UPDATE` on the checkout row stops *two different webhooks* from racing on the same checkout simultaneously. The row lock ensures only one worker proceeds. The second waits, then sees the checkout is already paid, and exits. Different events cannot race to the same outcome.

Both guards run inside a single database transaction. If anything fails mid-transaction, everything rolls back — no partial state, no inventory deducted without an order.

What breaks without these guards: Both workers read reserved, both deduct inventory, both create orders. Database commits both. Inventory goes to $N-2$. Two confirmation emails sent for one unit. This is the exact incident FlashCart already shipped.

8. What Not To Build

Not In V1	Why Not Now	Risk Of Building It Now
Async webhook queue	No evidence of gateway timeout problem. Team cannot operate it under drop load in one week.	Events silently drop or double-process. Invisible failures, harder to debug than synchronous handlers.
Microservices split	Not requested. Adds network hops and distributed transaction complexity.	Checkout and inventory become eventually consistent. More race conditions, not fewer.
Redis inventory cache	Cache drifts from DB truth under drop load.	Customers pay for inventory already gone. Cache becomes the new source of negative inventory.

9. Evidence Needed Before Trust

Claim	Evidence Needed
Queue prevents gateway timeouts	Production logs showing p95 webhook processing above 3 seconds during a real drop. Below that, synchronous handling is safer and simpler.

payment_id is sufficient for idempotency	Gateway documentation confirming no two concurrent deliveries share a payment_id, plus a load test with concurrent webhook replays under real drop concurrency.
Retry storm is safe under peak load	Connection pool metrics showing retries do not exhaust DB connections during a drop. Test with realistic concurrency numbers from previous drops.
Out-of-order events are rare enough to ignore	Official gateway documentation on delivery guarantees. Without it, out-of-order must be treated as normal behaviour, not an edge case.

10. Final Founder Recommendation

Decision: Block AI plan and replace with safer v1.

Event storage and retries are real ideas worth keeping — with the fixes described above. But Recommendations 5 and 6 will reproduce the double-order and negative-inventory incidents that caused the last public apology. The plan does not separate inventory reservation from payment confirmation — the core mistake behind both previous drops.

The safer v1 needs three database changes: reservation column on checkouts, UNIQUE(event_id) on payment_events, cron job for expiry. Buildable in one week. Recommendation 3 ships once the constraint is confirmed in production.

Do not ship Recommendations 5 or 6 as written. They sound correct. They use real vocabulary. They will cause real failures.

11. Final One-Liner

AI sounded correct, but it missed that idempotency needs two separate DB-level guards — UNIQUE(event_id) to stop the same webhook firing twice, and SELECT ... FOR UPDATE to stop two different webhooks racing on the same checkout — and that failed → paid requires human review not auto-processing — so I changed the design to reserve inventory at checkout start, enforce both guards inside a single transaction, and block any state override after a checkout has already failed.